

Lab Exercises on Multithreading in C# and .NET

Overview

These lab exercises are designed for intermediate .NET learners to understand and apply multithreading concepts using realistic, performance-oriented scenarios.

The labs focus on:

- CPU-intensive processing
 - Performance optimization
 - Task Parallel Library (TPL)
 - Thread synchronization
 - Shared resource management
 - Producer-consumer pattern
 - Parallel processing
 - Race conditions and deadlocks
-

Learning Objectives

By completing these labs, learners will be able to:

- Understand threading basics in .NET
 - Identify CPU-bound vs I/O-bound operations
 - Use `Thread`, `Task`, and `Parallel`
 - Improve performance using multithreading
 - Prevent race conditions
 - Synchronize shared resources safely
 - Use concurrent collections
 - Build thread-safe applications
-

Recommended Sequence

Lab	Topic	Difficulty
1	Sequential vs Multithreaded Processing	Easy
2	Parallel Data Processing	Easy-Medium
3	Image Processing Simulation (CPU Intensive)	Medium

Lab	Topic	Difficulty
4	Data Synchronization & Race Conditions	Medium
5	Thread-Safe Bank Account	Medium
6	Producer-Consumer Order Processing	Medium-Hard
7	Log Processing Engine	Hard
8	Deadlock Identification & Fix	Hard

LAB 1 — Sequential vs Multithreaded Processing

Scenario

A reporting system processes millions of transaction records to calculate tax summaries. Currently, the system is very slow because everything runs sequentially.

You need to improve performance using multithreading.

Step 1 — Create Sequential Version

Requirements

Generate 5 million random numbers and calculate:

- Sum
 - Average
 - Max
 - Min
-

Starter Code

```
using System.Diagnostics;

List<int> numbers = new();

Random random = new();

for (int i = 0; i < 5_000_000; i++)
{
    numbers.Add(random.Next(1, 1000));
}
```

```
Stopwatch sw = Stopwatch.StartNew();

long sum = 0;

foreach (var number in numbers)
{
    sum += number;
}

double average = sum / numbers.Count;

Console.WriteLine($"Sum: {sum}");
Console.WriteLine($"Average: {average}");

sw.Stop();

Console.WriteLine($"Time Taken: {sw.ElapsedMilliseconds} ms");
```

Tasks

1. Run the program
 2. Record execution time
 3. Increase data size to 10 million
 4. Observe CPU usage
-

Step 2 — Improve Using Parallel.For

Requirements

Split the work using `Parallel.For`.

Implementation

```
using System.Collections.Concurrent;
using System.Diagnostics;

List<int> numbers = new();

Random random = new();

for (int i = 0; i < 5_000_000; i++)
{
    numbers.Add(random.Next(1, 1000));
}

long total = 0;

object lockObject = new();
```

```
Stopwatch sw = Stopwatch.StartNew();

Parallel.For(
    0,
    numbers.Count,
    () => 0L,
    (index, state, localSum) =>
    {
        localSum += numbers[index];
        return localSum;
    },
    localSum =>
    {
        lock (lockObject)
        {
            total += localSum;
        }
    });

sw.Stop();

Console.WriteLine($"Total: {total}");
Console.WriteLine($"Time: {sw.ElapsedMilliseconds} ms");
```

Discussion Questions

1. Why is parallel processing faster?
 2. What is CPU-bound work?
 3. Why do we need `lock`?
 4. What happens if lock is removed?
-

LAB 2 — Parallel Image Processing Simulation

Scenario

A media company processes uploaded images by applying filters. Each image transformation is CPU-intensive.

You need to process multiple images simultaneously.

Requirements

Simulate image processing using expensive mathematical calculations.

Sequential Version

```
void ProcessImage(int imageId)
{
    Console.WriteLine($"Processing Image {imageId}");

    double result = 0;

    for (int i = 0; i < 1000000000; i++)
    {
        result += Math.Sqrt(i);
    }

    Console.WriteLine($"Completed Image {imageId}");
}

Stopwatch sw = Stopwatch.StartNew();

for (int i = 1; i <= 5; i++)
{
    ProcessImage(i);
}

sw.Stop();

Console.WriteLine($"Sequential Time: {sw.ElapsedMilliseconds}");
```

Task

Convert this program using:

- Task
- Task.WhenAll

Expected Learning

- Parallel execution
 - Task scheduling
 - CPU utilization
-

Suggested Solution

```
List<Task> tasks = new();
```

```
for (int i = 1; i <= 5; i++)
{
    int imageId = i;

    tasks.Add(Task.Run(() => ProcessImage(imageId)));
}

await Task.WhenAll(tasks);
```

LAB 3 — Prime Number Analyzer (CPU Intensive)

Scenario

A cybersecurity application validates encryption keys by checking prime numbers.

Prime calculation is CPU-intensive.

Requirements

Find all prime numbers between:

1 → 10,000,000

Step 1 — Sequential Version

```
bool IsPrime(int number)
{
    if (number < 2)
        return false;

    for (int i = 2; i <= Math.Sqrt(number); i++)
    {
        if (number % i == 0)
            return false;
    }

    return true;
}

List<int> primes = new();

for (int i = 1; i <= 1000000; i++)
{
    if (IsPrime(i))
    {
        primes.Add(i);
    }
}
```

```
}  
}
```

Step 2 — Parallel Version

Use:

```
Parallel.ForEach()
```

Challenge

1. Measure performance improvement
 2. Use `ConcurrentBag<int>`
 3. Compare with normal `List<int>`
-

LAB 4 — Race Condition and Synchronization

Scenario

An e-commerce system updates product inventory during flash sales.

Multiple threads update the same stock quantity.

Problem

```
class Inventory  
{  
    public int Stock = 1000;  
}
```

100 customers simultaneously buy products.

Unsafe Version

```
Inventory inventory = new();
```

```
Parallel.For(0, 100, i =>
{
    inventory.Stock--;
});

Console.WriteLine(inventory.Stock);
```

Observation

Expected:

900

Actual:

Different values each run.

Why?

Race condition.

Fix Using lock

```
class Inventory
{
    private readonly object _lock = new();

    public int Stock = 1000;

    public void Purchase()
    {
        lock (_lock)
        {
            Stock--;
        }
    }
}
```

Challenge

Implement using:

- lock
- Interlocked

- SemaphoreSlim

Compare performance.

LAB 5 — Thread-Safe Bank Account

Scenario

Multiple ATM machines access the same bank account simultaneously.

Requirements

Create:

```
class BankAccount
{
    public decimal Balance { get; private set; }

    public void Deposit(decimal amount)
    {
    }

    public void Withdraw(decimal amount)
    {
    }
}
```

Tasks

1. Simulate 1000 parallel deposits
 2. Simulate withdrawals
 3. Prevent negative balance
 4. Make account thread-safe
-

Concepts Covered

- Locking
- Critical section
- Atomic operations
- Data consistency

LAB 6 — Producer Consumer Pattern

Scenario

An online food delivery system receives orders continuously.

- Producers = Customers placing orders
 - Consumers = Kitchen preparing food
-

Requirements

Use:

`BlockingCollection<Order>`

Producer

```
BlockingCollection<Order> orders = new();

Task producer = Task.Run(() =>
{
    for (int i = 1; i <= 100; i++)
    {
        orders.Add(new Order { Id = i });

        Console.WriteLine($"Order {i} placed");
    }

    orders.CompleteAdding();
});
```

Consumer

```
Task consumer = Task.Run(() =>
{
    foreach (var order in orders.GetConsumingEnumerable())
    {
        Console.WriteLine($"Preparing Order {order.Id}");

        Thread.Sleep(500);
    }
});
```

Challenge

1. Add multiple consumers
 2. Measure throughput
 3. Add cancellation token
 4. Add order priority
-

LAB 7 — Parallel Log File Analyzer

Scenario

A cloud application generates huge log files.

You need to:

- Read logs
 - Parse errors
 - Generate summary reports
-

Requirements

Generate fake logs:

```
INFO
WARN
ERROR
DEBUG
```

Tasks

1. Read logs sequentially
 2. Process using parallel tasks
 3. Count ERROR occurrences
 4. Store results thread-safely
-

Concepts Covered

- ConcurrentDictionary

- Parallel LINQ (PLINQ)
- Task Parallelism

Sample PLINQ

```
var errorCount = logs
    .AsParallel()
    .Count(x => x.Contains("ERROR"));
```

LAB 8 — Deadlock Detection and Prevention

Scenario

A payment system locks:

- Customer Account
- Merchant Account

Incorrect locking causes deadlocks.

Deadlock Example

```
object lockA = new();
object lockB = new();

Task t1 = Task.Run(() =>
{
    lock (lockA)
    {
        Thread.Sleep(100);

        lock (lockB)
        {
        }
    }
});

Task t2 = Task.Run(() =>
{
    lock (lockB)
    {
        Thread.Sleep(100);

        lock (lockA)
```

```
        {  
        }  
    }  
});
```

Tasks

1. Reproduce deadlock
 2. Identify why it happens
 3. Fix by consistent lock ordering
 4. Fix using timeout locks
-

Advanced Topics for Exploration

Threading APIs

API	Use Case
Thread	Low-level threading
Task	Recommended modern async work
Parallel.For	CPU-bound loops
PLINQ	Parallel LINQ
async/await	I/O-bound async work

Performance Measurement Exercise

Requirements

Compare:

- Sequential
- Parallel.For
- Task.Run
- PLINQ

Measure:

- Execution time
 - CPU usage
 - Memory usage
-

Suggested Metrics Table

Technique **Time (ms)** **CPU Usage** **Complexity**

Sequential

Parallel.For

Task.Run

PLINQ

Real-World Discussion Topics

When NOT to Use Multithreading

- Small datasets
 - Heavy locking
 - I/O-bound operations
 - Shared mutable state
-

Common Problems

Problem	Description
Race Condition	Multiple threads modify same data
Deadlock	Threads waiting forever
Starvation	Thread never gets CPU
Context Switching	Too many threads reduce performance

Best Practices

Recommended

- Prefer `Task` over `Thread`
 - Use immutable objects
 - Minimize locks
 - Use concurrent collections
 - Measure performance before optimizing
-

Mini Capstone Challenge

Build a Parallel Report Generator

Features

- Read large sales data
- Process regions in parallel
- Generate summaries
- Export reports

Requirements

Use:

- Task
- Parallel.ForEach
- ConcurrentDictionary
- CancellationToken
- Stopwatch

Final Reflection Questions

1. What types of applications benefit most from multithreading?
2. Difference between concurrency and parallelism?
3. Why can multithreading sometimes reduce performance?
4. What is thread safety?
5. What is the role of the ThreadPool?